

Relation to Graph Concepts

Rajiv Sambasivan

September 6, 2025

1 Introduction

This document provides an overview of the concepts related to the transformation of relations into graph structures. We begin by defining the key terms and concepts that will be used throughout the discussion.

1.1 The Relational Model

[4] introduced the relational model, which is the logical foundation for relational databases. In this model, data is represented as relations (tables), where each relation consists of tuples (rows) and attributes (columns). An algebraic framework for manipulating these relations is provided by relational algebra, which includes binary operations (operate on two relations) such as join and union, as well as unary operations like selection and projection (operate on a single relation). The relational model provides a mathematical framework for representing and querying data.

1.2 The Entity-Relationship Model

The entity-relationship (ER) model, introduced by [2] is a framework for describing the *semantic* data abstractions and their relationship with each other. For example, if your use case is about how students in a university register for courses, then *students*, *course*, *registration* might be a set of data abstractions. The *registration* may connect a student with a course for a particular semester. From a *data representation* point of view, all semantic abstractions may be represented as relations.

1.3 The Entity-Relationship to Relational Schema Mapping

There exist guidelines and a body of knowledge around mapping an *Entity-Relationship Model* to a *Relational Schema*. See [6], for example.

1.4 Use Case Data

Your *Use Case Data* is what you want to convert to a graph. The data for your use case is a set of extracts that have a semantic and database (data representational mapping). As a modeler, you are expected to know:

- The semantic abstractions relevant to your use case. These are the *entities* in your data extract. You should make note of this.
- The *semantic relationships* in your usecase. You should understand how *entities* in your usecase are related.
- The *semantic to extract mapping* for your usecase. You should know which extract file correspond to a particular entity or relationship. The extracts are what you get from the relational databases supplying the data for your use case. As an extension, we can look at connectors that fetch data from source relational systems, but for now we will assume you have file extracts of relations and entities.

1.5 Analysis

Given a use case, you first need to understand the characteristics of the data as it relates to your use case. In large datasets, there is often heterogeneity in the data. Heterogeneity means that the data is not uniform. For example, in a financial dataset, you may have data from different lines of business, different product categories, or different geographic regions. Each of these segments may have different characteristics that need to be accounted for in your analysis. Heterogeneity is discussed in detail in [2.2](#).

You may need several iterations of analysis to understand the data. You will need to evaluate different ways to assess the data from the perspective of your use case. Your use case may be associated with many metrics. For example, a revenue management use case may be associated with metrics such as revenue, profit, customer satisfaction, and market share. You will need to evaluate different ways to assess the data from the perspective of these metrics. A health care use case may be associated with metrics such as patient outcomes, cost of care, and patient satisfaction. You will need to evaluate different ways to assess the data from the perspective of these metrics.

For each perspective also need to to evaluate different modeling approaches to see which one works best for your use case. Making an informed choice usually requires iterations to evaluate different modeling approaches from the perspective of desired performance metrics of your use case. Defining the performance metrics for your use case is an important step in this process.

The analysis step is the core focus of this document.

1.6 Modeling

Once you have a good understanding of the modeling approaches, heterogeneity in the data, and the performance metrics for your use case, you can start to build

models. Evaluation of the modeling approaches is also part of the analysis step.

Once the approach is selected, you can start to build models. Operationalizing the model and monitoring the model is also part of the modeling step. This is typically done by an MLOps team. This is outside the scope of this document.

2 Guidelines for Implementation

The implementation consists of an initial assessment of the data, analysis for heterogeneity and a graph mapping. The guidelines for performing these steps are given below.

2.1 Initial Assessment

For the initial assessment, use a `jupyter notebook` to do the following:

- Inspection of Dataset: This is a quality assessment of the extracts you received for the entities and the relations. For each entity, verify that you have the required attributes in the extract. Make note of the desired attributes, you will need this to define the workflow to process the dataset.
- A *critical* point to note and understand at this point is the nature of analysis the use case is performing, i.e., are you doing a *Crosss-Sectional*, *Longitudinal* or *Panel* study.
- Obtain domain expertise about sources of heterogeneity in the data. You can also use data-driven techniques to identify sources of heterogeneity. For example, if your use case is going to translate to a supervised learning task, then you can use feature selection techniques to identify the most relevant features for the outcome of interest. You can then cluster the data based on these features and profile the resulting groups to understand the relationship between the features and the outcome. The cluster then becomes a source of heterogeneity. If you are doing a longitudinal study, then you can use change point analysis to identify sources of heterogeneity over time. See [10] for details.
- Understand and document the business rules that must be satisfied in your final data representation. For example, if you have a finish time and a start time, you may want to verify that the finish time is no earlier than the start time. On financial data, signs, value-ranges etc., may need verification. Translate each rule to appropriate code. Sure there are data quality tools, but since there are only 5 entities per use case, the overhead of a data quality tool may not be worth it. Make note of how the business rules must be verified and how non-conforming records can be flagged, logged and dropped from the dataset.

- If you need numeric features for heterogeneity analysis, then you might want to featurize the dataset. If, for example you need to use clustering to group data into similar subsets, then you need to perform featurization where data for each entity chain is completely numeric. If you want to rely on segmentation, which translates to a group-by operation, to identify heterogeneity, you don't need to featurize at this point. So when you want to featurize a dataset will depend on how you want to do heterogeneity analysis.

The note comments in each of the steps above will be used to guide further processing.

2.2 Heterogeneity Analysis

The dataset that you created at the end of the previous step will be used for heterogeneity analysis. See [10] for details. The point about being clear about the type of analysis you want to perform for your use case will help with the analysis of heterogeneity.

In a *cross-sectional* analysis, sources of heterogeneity may be identified through domain expertise or by applying data-driven techniques. For instance, business datasets often exhibit heterogeneity across dimensions such as line of business, product category, or geographic region. To systematically uncover these sources, you can cluster the dataset based on variables that influence the outcome of interest. For example, if revenue is the key metric, performing feature selection—such as regressing revenue on various features—can help identify the most relevant variables. Clustering the data using these variables and profiling the resulting groups enables you to pinpoint and understand the main sources of heterogeneity.

In a *longitudinal analysis*, heterogeneity occurs over time. Tracking changes with respect to time using *change point analysis* can tell you when and how many changes have occurred in the period that you are monitoring. Of course, you will need collaboration with domain experts to validate the number and points of change. See [11] for an example. A reasonable approach to arrive at a set of candidate change points is to use an appropriate smoothing technique (a moving average filter, or another profiling method such as the *Matrix Profile*[12]), set up a decomposition[3] of the variable that is changing over time and then histogram the components of the decomposition. By counting the modes in each component of the decomposition and then adding them up, we get a baseline for the number of possible changes in the variable over time. Such a baseline can be discussed with domain experts and refined. The results from such a discussion can inform the *change point* detection algorithms.

In *Panel Data*, you need to account for sources of variation you have observed with both *cross-sectional* and *longitudinal* data.

Based on the principles discussed, partition the dataset from the previous step into meaningful subsets. The approach you use to create these partitions should align with the type of data analysis being performed—whether cross-

sectional, longitudinal, or panel—and the specific objectives of your use case. For example, in supervised learning tasks with imbalanced data, you may need to apply specialized partitioning strategies compared to those used for balanced datasets. The primary goal is to simplify the analysis within each partition, which can be achieved by ensuring that each subset is as homogeneous as possible with respect to the outcome of interest. In supervised learning, this might involve creating partitions where simple models perform well; in unsupervised tasks such as dimensionality reduction, you might first cluster the data and then apply dimensionality reduction techniques within each cluster. Developing detailed guidelines for various scenarios is planned for future work. This step requires solid data analysis skills, and consulting with domain experts is highly recommended to ensure effective partitioning.

2.3 Graph Mapping

The first challenge you are likely to face is the decision about how to map the source entities and relationships to a graph structure. The source system we are dealing with is a relational database. The assumption here is that we are dealing with an enterprise use case where there is usually a pivotal entity that connects to other entities. For example, in a retail use case, the pivotal entity may be a *transaction* that connects to other entities such as *customer*, *product*, *store*, *time*. In a health care use case, the pivotal entity may be a *patient* that connects to other entities such as *doctor*, *diagnosis*, *treatment*, *time*. In a manufacturing use case, the pivotal entity may be a *work order* that connects to other entities such as *machine*, *operator*, *time*, *product*.

The stage of analysis is also a consideration in how to map the source entities and relationships to a graph structure. If you are trying to understand the behavior of the pivotal entity, then you may want to use a graph structure that is centered around the pivotal entity. This is a *homogeneous* graph structure where the nodes are the same type of entity, in this case the pivotal entity. Such a graph structure is useful in identifying heterogeneity in the behavior of the pivotal entity. Such a graph structure is also useful in identifying patterns in the behavior of the pivotal entity. If you are trying to understand the relationship between the pivotal entity and other entities, then you may want to use a *heterogeneous* graph structure where the nodes are different types of entities. Such a graph structure is useful in identifying heterogeneity in the relationships between the pivotal entity and other entities. Such a graph structure is also useful in identifying *normal* and *anomalous* behavior of the pivotal entity. Understanding the behaviour of the pivotal entity is a major application of graph analysis in enterprise use cases. Please see [7] and [8] for a detailed discussion. When you want to connect two entities from the perspective of identifying heterogeneity in the relationship between the two entities (normal and anomalous behavior can be regarded as one example of this), then using a similarity measure or a distance metric to connect the two entities is a reasonable approach.

Often we are also interested in understanding a specific binary relationship

between two entities. For example, in a retail use case, we may be interested in understanding the relationship between *customer* and *product*. In a health care use case, we may be interested in understanding the relationship between *patient* and *treatment*. In a manufacturing use case, we may be interested in understanding the relationship between *machine* and *operator*. In such cases, we may want to use a graph structure that is centered around the two entities of interest. This is also a heterogeneous graph structure where the nodes are different types of entities. Specifically, this type of graph is a *bipartite* graph. Understanding how a pivotal entity interacts with another entity is a common analysis task in enterprise use cases.

Finally, once we have identified heterogeneity in the data, we may be interested in predicting:

- The property of an specific entity
- The property of a relationship between two entities
- The property of a collection of entities

For these problems, we can map the source entities and relationships to a graph structure that replicates the entity relationships in the source system. This is a heterogeneous graph structure where the nodes are different types of entities. The edges are the relationships between the entities. The tasks described above correspond to *node classification*, *link prediction* and *graph classification* tasks in the graph learning literature. See [13] for a detailed discussion.

In summary, three analysis tasks are common in enterprise use cases:

- Understanding the behavior of a pivotal entity. This is a homogeneous graph structure where the nodes are the same type of entity, in this case we need to pick the pivotal entity and a suitable similarity or distance metric to connect the nodes.
- Understanding a specific binary relationship between two entities. This is a heterogeneous graph structure where the nodes are different types of entities. Specifically, this type of graph is a *bipartite* graph. The input to this analysis is the two entities of interest and the relationship between them.
- Predicting the property of an specific entity, the property of a relationship between two entities or the property of a collection of entities. In this case the graph structure replicates the entity relationships in the source system. This is a heterogeneous graph structure where the nodes are different types of entities. The edges are the relationships between the entities.

The analysis themes discussed above are by no means exhaustive. These are just the common themes that we have observed in enterprise use cases. Graphs have been used in many other ways to solve a variety of problems in engineering and science. For a discussion of other applications, for example to science and engineering, please see [5].

In this document, we are primarily interested in transforming relational models to graphs. The details of developing machine learning models on graphs is outside the scope of this document. For this, please see [1].

3 Knowledge Graph Construction

A review of the document so far will show that there are several steps in the production of a graph from relational data where modeler has to make decisions and choices. The modeler has to understand the use case, the data, the sources of heterogeneity in the data and the type of analysis that is required for the use case. The modeler also has to understand the different types of graph structures and their suitability for different types of analysis.

Knowledge graphs make it possible to capture the decisions and choices made by the modeler in a structured way. A knowledge graph is a graph that represents knowledge in a structured way. The nodes in the graph represent entities, concepts, or ideas. The edges in the graph represent relationships between the nodes. The knowledge graph can be used to capture the decisions and choices made by the modeler in a structured way. The knowledge graph can also be used to capture the reasoning behind the decisions and choices made by the modeler. This makes it possible to reproduce the analysis and modeling steps taken by the modeler.

Knowledge Management for Data Science (KMDS) is a framework for capturing the decisions and choices made by the modeler in a structured way. The framework is based on the principles of knowledge management and data science. The framework provides a set of tools and techniques for capturing, storing, and sharing knowledge in a structured way. The framework also provides a set of tools and techniques for analyzing and modeling data in a structured way. The framework is designed to be flexible and adaptable to different types of use cases and data sources. The framework is implemented in Python and is available as an open source project on GitHub[9]. However, the integration of KMDS with the steps outlined in this document is a work in progress. KMDS is mentioned here to highlight the importance of capturing the decisions and choices made by the modeler in a structured way.

References

- [1] Ines Chami et al. “Machine learning on graphs: A model and comprehensive taxonomy”. In: *Journal of Machine Learning Research* 23.89 (2022), pp. 1–64.
- [2] Peter Pin-Shan Chen. “The entity-relationship model—toward a unified view of data”. In: *ACM transactions on database systems (TODS)* 1.1 (1976), pp. 9–36.

- [3] William S Cleveland. “LOWESS: A program for smoothing scatterplots by robust locally weighted regression”. In: *The American Statistician* 35.1 (1981), p. 54.
- [4] Edgar F Codd. “A relational model of data for large shared data banks”. In: *Communications of the ACM* 13.6 (1970), pp. 377–387.
- [5] Diane J Cook and Lawrence B Holder. *Mining graph data*. John Wiley & Sons, 2006.
- [6] Ramez Elmasri and Shamkant B Navathe. *Fundamentals of database systems seventh edition*. pearson, 2016.
- [7] al. Faloutsos Christos et. *User Behavior Modeling Part*. Aug. 3, 2025. URL: <https://www.youtube.com/watch?v=fMHZt4UStUg&list=PL-lbroKJyNLCAM85ENZ8g2DEiG1e5TQEo&index=1>.
- [8] al. Faloutsos Christos et. *User Behavior Modeling Part*. Aug. 3, 2025. URL: https://www.youtube.com/watch?v=eI-hmySej_w&list=PL-lbroKJyNLCAM85ENZ8g2DEiG1e5TQEo&index=2.
- [9] Rajiv Sambasivan. *GitHub - rajivsam/KMDS: Source for KMDS — github.com*. <https://github.com/rajivsam/kmds>. [Accessed 06-09-2025].
- [10] Rajiv Sambasivan. *Heterogeneity in Modeling*. Aug. 3, 2025. URL: https://rajivsam.github.io/r2ds-blog/posts/heterogeneity_in_models.
- [11] Rajiv Sambasivan. *Understanding Coffee Prices*. https://rajivsam.github.io/r2ds-blog/posts/markov_analysis_coffee_prices/. Accessed: 2025-06-21. 2025.
- [12] Chin-Chia Michael Yeh et al. “Matrix profile I: all pairs similarity joins for time series: a unifying view that includes motifs, discords and shapelets”. In: *2016 IEEE 16th international conference on data mining (ICDM)*. Ieee. 2016, pp. 1317–1322.
- [13] Jie Zhou et al. “Graph neural networks: A review of methods and applications”. In: *AI open* 1 (2020), pp. 57–81.